

Store Locator Software

Capstone Report
Presented to the
Faculty of CST 499 at
California State University, Monterey Bay

In Partial Fulfillment
Of the Requirements for the Degree
Bachelor of Science
In
Computer Science

By
Bijan Pourazari
Summer 2021

Executive Summary of Report

Store Locator Software

By

Bijan Pourazari

Bachelor of Science in Computer Science

California State University Monterey Bay, 2021

The purpose of the capstone project is to develop a software component that will provide an easy to use development process for Store Locator functionality that meets all the latest up to date technology in interactive maps. The goal is to minimize the development time that developers will need to spend in creating their desired functionality. In order to achieve this goal, an NPM package will be deployed that developers can download and install via NPM and use to develop in their Angular applications.

In person shopping is still a highly important avenue for businesses. This is despite the growth of online shopping and eCommerce. Businesses still need a way to get customers into stores. Customers need a way of locating their nearest stores. Many businesses are lacking in the way they provide for this need. Many do not have interactive locator maps on their sites. This project aims to solve this problem by providing a solution for developers to streamline their development of a Store Locator in their online presence.

development of a Store Locator in their online presence.

Table of Contents

I Introduction/Discussion	5
Project Name and Description	5
Problem	5
Solution	6
Project Goals and Objectives	7
Community and Stakeholders	8
Evidence That The Project Is Needed	9
Feasibility Discussion	10
II Design Requirements	11
Functional Decomposition of Project	11
Selection of Design Criterion	12
Final Deliverables	13
Approach/Methodology	13
Ethical Considerations	14
Legal Considerations	15
III Project Scope	15
Timeline/Budget	15
Usability Testing/ Evaluation	17
Final Implementation	18
Discussion	20
References	23
Appendix A	24

Introduction/Discussion

Project Name and Description

The capstone project that has been underway is a Store Locator project. The project is designed to benefit anyone that utilizes the web for eCommerce and shopping, as well as any businesses that have an online presence with the need to get their customers into their stores.

Problem

The motivation behind this project was to solve a problem in the business world, and to do so in an efficient way that saves development time. The specific problem in mind was the need for business owners to get customers into their stores. In today's day and age, online shopping is a dominant force in eCommerce. Many potential customers shop online and have their purchased products delivered straight to their door. Despite this, in person shopping is still very relevant and there are even some industries where in person shopping is required due to the nature of the industry. In these cases, it is incredibly important for online retailers to provide to the customer the location of their stores.

Going further, it has been shown that there is a need for these online stores to have the most up to date technology utilized in their Store Locators. As it stands, this is not always the case. Businesses can be utilizing very outdated technology and not providing the best interactive experience to their customers. In some cases, businesses may be even completely failing to provide this service on their websites entirely.

Just as importantly, there is a development cost and time needed to provide the missing solutions that customers expect. Part of the reason that businesses may not have the most up to

date interactive map technology utilized in their store locators is the development cost and time. This is a problem that this capstone project aims to address.

Solution

The proposed solution to the problem was a Store Locator software that would provide an interactive experience for users. At first, during the development of the idea for this project, it was the intention to provide a solution for a business to use. As time went on, the idea evolved into producing a generic software solution that any business could have its Software Developers incorporate and use in their development process. This solution idea evolved to where it was designed to be as helpful to as many companies as possible, and an open source software solution using the NPM package registry became the plan.

What is very interesting about this proposed solution is that it doesn't just solve the direct problem of needing a store locator. It also helps solve the problem of development times being too long and costs being too high. Part of this motivation came from the student developer's own experience working as a Software Engineer. Working with useful software components in his own work was the main inspiration for this project. Going even further, this software that is being developed for the capstone project is also going to be free and open source, instead of the paid software the student developer worked with. That means more people will be able to contribute to the project in the long term, hopefully to keep it working for years to come.

The main implementation of the software that has been in development now is leveraging the Google Maps API for providing the interactive map solution. The main benefit of using Google Maps API is the quality of the interactive map that is provided, and the ease in coding. It is a relatively straightforward API to use, although there is some overhead in needing to create a

Google Cloud Platform account in order to gain access to an API key. That being said, the benefits outweigh any disadvantages. Upon completion of the project, developers will be able to download and use the software component in their Angular applications and have a working store locator that is implemented with the most up to date Google Maps API technology.

Project Goals and Objectives

The main goals of the project were outlined to be the following: Finalize the tech stack to use; Learn how to create and deploy NPM packages; Design the look of the application; Complete the final software; Deploy final software to NPM. Within each of these goals, there were identified objectives that needed to be achieved.

For the goal of Finalizing the tech stack to use, there were two objectives identified. These were researching the available tech stacks for 1 hour daily, and to code sample projects weekly until the final capstone class was to start. These objectives were intended to be completed before the class even started, so that the student developer would be prepared for the final project.

For the second main goal of learning how to create NPM packages, the objectives that were identified were to study the NPM documentation 1 hour daily until the material is understood, watch tutorial videos for 1 hour daily until the material is understood, and to code sample npm packages and upload them to npm weekly.

For the third main goal of designing the look of the application, there were two key objectives identified: creating a mock layout, and making revisions to the layout. Breaking these down even further, it was identified that it would be important to create mock software, benchmarking designs, and laying out the web page.

For the final main goal of completing the final software, there were three key objectives identified: coding the layout of the application, coding the interactive map, and coding the data sources. Going even further, the coding of the interactive map was broken down further into the objectives of coding the markers on the map, coding the geolocation, and coding the search.

The final goal of the ultimate deployment of the packages to npm had two main objectives identified: testing and the actual deployment.

Community and Stakeholders

The student developer identified the main stakeholders of the project to be the businesses that will be utilizing the capstone project's software components. The community that would benefit from the software was identified to be the software developers who will utilize the components in their daily software development, as well as the users, who are the customers that will be searching for products to buy and need to find stores on the website.

This project will make a difference in the lives of the developers by making their development process easier and straightforward. Instead of having to code a store locator from scratch themselves, the developer will be able to leverage this project's software to streamline their development process.

As it relates to the main stakeholders, the student developer identified a key need. As shown by statistics done from studies (Wandosell et al), not all businesses that have online presences are utilizing store locators, and of the ones that do, not all are utilizing the latest technology in interactive maps. The student developer identified this as the key problem that the stakeholders had. If potential customers are interested in making a potential purchase, and they are not able to find the nearest store, then the stakeholders could be missing out on potential

customers. Furthermore, when customers are shopping they expect a certain standard when it comes to technology. For example, most customers today shopping on the internet expect websites to be mobile friendly. Any site that does not meet their standards could lower their impression of the brand and cause them to shop elsewhere. These were the factors the student developer weighed when deciding that this project was a valuable one.

When it comes to the community of developers, the student developer took motivation from his own personal experience working with software components in his day to day work life. The student developer had recently worked with a well designed software component that made his life easier during his work. It streamlined the development process and made things go smoother. This was a big factor in the decision to go forward with this project. He believed that it would be a huge benefit to other developers and increased the motivation for working on the project.

Evidence that the Project is Needed

According to the study done by Wandosell et al, physical retail is the most used operation format in the world. Furthermore, while doing their study on store locators, they discovered that a large percentage of websites for businesses that had store locators on them were lacking in having the most up to date interactive map technology. According to Wandosell et al, “...most large scale retail firms included store locator tools on their websites, but the type and format of presentation varied significantly among them... most websites included interactive web maps... but some retail websites still included this information using non-interactive tools.” This is essentially the main reason that the student developer identified this project as being needed by the community and stakeholders. Generally, consumers demand that they have the highest

quality and most up to date technology when they are shopping. When presented with outdated tech, they can be turned off from the business and have a negative impression of the brand, and thus choose to shop elsewhere. And since there is a significant portion of businesses who are still utilizing non-interactive tools, there is a need that this project can satisfy.

Feasibility Discussion

The results of the Environmental Scan of the proposal showed that there were certainly places where Store Locators were also being used online. In fact, there are many online retailers that are already utilizing online Store Locators. This project does have many similarities to the store locators that are already being utilized, especially in the sense that there will be an interactive map in the application. After doing the Environmental Scan, the student developer decided that even though there already existed instances of store locators already being used, that there was still enough of a way for this project to stand out. This is because this project proposed a generic solution that would be packaged as a reusable component so that it could provide a benefit to many Software Developers.

The main difference between this project and those that already exist, is that the ones that already exist are single, one-time solutions. They are custom solutions that while may leverage similar API's, for example the Bing Maps API or the Google Maps API, they are still not reusable and had to be built from the ground up. That is the main reason why the student developer chose to go ahead with this project. It was the hope that it would be a reusable and useful solution that many will find useful.

According to Wandosell et al., it was concluded that retailers should use interactive maps in their store locator software. This is because it provides the best connection between the retailer

and the consumer. This is the main way that the student developer aims to have the project stand out among the rest. It will be of great benefit to all the businesses that are still using static maps and non-interactive technology for their store locators. For the businesses that have no store locators, this project will stand out because it will allow them to develop one and add it to their online presence.

To summarize, this project is certainly not an entirely new idea. There are many already existing store locators. Many businesses already provide ways for customers to find and locate their stores. However, a lot of these solutions are not utilizing the latest technology. The student developer felt that this indicated that there was a good opportunity to help out with this need. Finally, the software will be wrapped as a reusable component, so it will have the added benefit of reducing development time. All of these factored together in making the project a very exciting one for the student developer.

PART 2

Design Requirements

Functional decomposition of project

For this project capstone, there are several key components outlined in the design. The project is fundamentally an application component designed to provide the functionality of a Store Locator on a website. The requirements of the design are that the store locator has an interactive map that users can use to view store locations in real time. The users are able to search by their current location, by other addresses, by state, and by zip code to return a list of stores within a certain search radius. Users are able to use a dropdown to select which way of search they plan to use, as well as use a dropdown to select the search radius.

Once these options have been selected, the user can then click the search button to execute the search. The search results are returned in a paginated side panel, and the map is populated with markers at their geolocations that the users can then click to view an infowindow with more information. The map also has its bounds re-centered around the newly returned markers, and zoomed in to the appropriate zoom level to fit all of the markers. The user can click the next button to page through the paginated results. As they move from page to page, the map is updated with new markers and re-centered accordingly. At this point, the user can view the relevant details for each store, such as their address, their phone number, etc. They can then click a directions link to be linked to google maps directions from their location to the store.

Selection of Design Criterion

The desired goals for performance are that the store locator is intuitive to use, and the process of searching for stores is clear and easy to do. Additionally, it is desired that the design of the application is laid out in a way that the user is comfortable with. The choice of the Google Maps API for the provider of the interactive map was made with this in mind, as many users will already be familiar with it.

As far as the software side of things, it is desired that the code is easy to understand and that developers will have no problem at all utilizing the component in their development process. Because the code is also going to be open source, it is the goal of the student developer that the source code is well organized and easy to read and modify if the developers would like to make their own modifications to the code.

Final Deliverables

The final deliverables of the project will be the uploaded npm package that can be downloaded by any developer and used in their Angular applications to provide a store locator solution for their websites. Additionally, for the purposes of the capstone festival, the student developer will provide a working example of the software on a sample “mock” website written in Angular that will showcase the project in a working scenario with real sample data. Additionally, the mock website will come with the documentation that will be used by developers as they are developing with the component. It will explain how to install the component, how to place it in their html, and how to provide it with the stores’ data and how that data will need to be formatted.

Approach/Methodology

The approach/methodology that was used by the student developer in the project was a similar approach to the development that he does in his day to day work life as a Software Engineer. The very initial methodology used was to research possible projects well before the capstone class even began. This was to ensure that the project selected would be both interesting to present to the professors while also having the ideal scope and size to fit into the 8 week class duration. Research began using simple online google searches looking for ideas. Once the Store Locator idea was developed, the student developer started to gain interest in the idea of creating a reusable solution that many developers could leverage to provide themselves with a store locator on their websites.

Research then began into how to provide such a reusable component solution, and in what programming languages this would best be implemented. Thus began testing and research with software components available on the web. This led in the direction of the npm package

manager and software registry. It presented itself as the most logical choice since it was both a free open source way to share Javascript code which is the language of the web. It is also in use by over 11 million developers worldwide. During the early approach to the project, the student developer discussed the idea with several other Software Engineers to see if they could provide any insight. They concluded that it was indeed a good idea to use npm.

At this point, the student developer had used npm to install components and use them in his projects for work. However, he had no experience creating one of his own. So the next steps were to do research into the difficulty of creating a package. There were many online resources for creating node packages, and the student developer located documentation on how to create and upload a package.

Once it was concluded that it was a reasonable approach to use npm, the development language that was chosen to use was Angular and Typescript. This was simply due to the experience that the student developer had in working with these technologies. With this in mind, the next steps were to begin designing the application, and learn how to work with the Google Maps API, which was what the developer decided would be the best way to provide the interactive map. From this point on development continued with creating the map, working with geolocation and geocoding API's, and coding the services for searching an array of stores by a search radius and displaying the results on the map.

Ethical Considerations

The main ethical considerations that were identified in the early development process were in relation to the use of the user's location information. The student developer believed and still believes that consumers have an expectation that their privacy is respected when they do online shopping and searching for stores to make purchases. It was decided that the application

would operate in a way that was very considerate of the user's privacy by making no use of their location other than in providing directions to the stores. The application does not store their location anywhere, such as in a database, and it utilizes their browser's options for declining the use of their current location. Once the geolocation request is made, the user is presented with an option of whether they would like to allow the browser to use their current location. They are then able to decide for themselves whether or not to opt into this option. This is the way the application stays considerate of the user's privacy.

Legal Considerations

The main legal considerations that were identified by the student developer were whether the software should be sold as a commercial product, or whether it would be released as open source. Furthermore, should the project have any type of warranties or disclaimers? Should it have any sort of assurance that the software will work? Because of the scope of the project and short time frame that the student developer had to finish the project, the final decision that was made was that the project would be released as open source software with the statement that it is being released "As Is". This means that it will come with no assurance that the product will work, that it comes without warranty, and that it frees the student developer from any type of liability.

PART 3

Timeline/Budget

Overall, the timeline went relatively smoothly. Most of the milestones were met in the time frame that was planned out. It was initially desired that there would be about a week of

extra time before the Capstone Festival to have the project entirely completed. In actual reality, in order to get an up and running prototype of the Store Locator that was running using an external library (which was the desired way that the package would be released), it took all the way until the night before the Capstone Festival. There were definitely headaches and struggles that were presented when working with converting the development code into a bundled library. Going into specifics, the development was done in an Angular workspace that contained a Single Page Application. All development was done here. This single page application included the necessary modules for serving a web application. A component was created in this development web application, and all of the google maps code for the map and geocoding was integrated here in addition to the layout of the application as well as a service handling the searching for stores based upon being located within a certain radius from the search location.

All of this code was working in the development environment in a timely manner. Based upon the timeline that was established during the proposal, it was desired that the coding of the interactive map and the coding of the search functionality would be completed in week 3 and in week 4 respectively. The goal was to have a working prototype ready for the testing that was to be done in week 5. This goal was met well in advance. The prototype was ready almost a full week in advance of the testing, and this left several weeks to respond to testing feedback, as well as finish up the coding. Some changes were made to the service handling search, as well as a new geocoding with autocomplete feature was added after testing. This new autocomplete feature is very useful for the user and was a great addition to the project, and even with making this unplanned addition coding was completely finished in week 5.

However, actually deploying the application as a library that could be uploaded to NPM was not completed until the night before the Capstone event in week 7. This was definitely

cutting it closer than was desired. A secondary way for developers to utilize the Store Locator component was implemented by utilizing Github as a way to upload the code for the component that developers could just download and use. In the end, however, the errors were fixed and a working prototype from a built library was completed before the capstone festival.

As far as the budget is concerned, the project was done all for free. It was budgeted that there would be no costs since everything that the project relies upon is free software. There is only one consideration for budget, which is that the users of the component will need to register with Google in order to enable the services that the component uses. They will receive an api key that they will use with the component, and they will receive pricing information for the services Google provides. So the component is not entirely free to use. The amount of money that the developers will pay to Google depends upon the amount of usage.

Usability Testing/ Evaluation

The usability testing that was conducted for the project was done with three testers. The desire was to test both the functionality of the store locator as well as the code implementation and ease of use for other Software Developers to use. With this in mind, two of the testers were Software Developers. The third was a Graphic Designer, who had experience with web layout design. This made the feedback from the testers very useful.

The testing was performed in person, face to face, or over a screen share format. The two in person testers were given control of the keyboard and mouse and allowed to test a prototype that was loaded with sample data. The testers began asking questions as they used the application, and the developer answered any questions that they had as they were testing.

For the screen share testing process, there was a difficulty in that the tester did not have control over the developers keyboard and mouse. In order to deal with this issue, the tester told the developer what to click on and what to type.

The results of the testing process was some very positive feedback over the functionality of the application. The design of the application was said to be very satisfactory. Everything was laid out in a way that was easy to understand for the users. The overall features of the application met the needs of what a potential customer would be looking for as they search for stores.

In regards to the code itself, the two Software Developers provided feedback onto potential improvements for the way the code was done. A code review was essentially done at this point in time, which was a big help. There was agreement that the library solution to package the component into a reusable application would be a very solid solution. There was feedback regarding how the Google Maps API was integrated with the application. There was also feedback regarding the handling of dependencies that the library would use. Interestingly enough, later on when there was difficulty in porting the working development code into a packaged library, some of the discussion that was done during the testing process was useful in figuring out the solution. Finally, some excellent suggestions for additional features and another related project regarding creating an application for entering stores and returning JSON files with the store information were the focal point of the conversation during testing.

Final Implementation

The final project is an Angular library that implements a store locator. An Angular Library is a reusable set of code that contains Angular components, services, and models that can be used again and again. The library has been packaged and deployed to NPM. The library contains the following files: a css file containing the layout information for the map; an html file

that contains the template for the page; a typescript file that contains the code for the component of the store locator; a typescript file containing the service that handles the searching of the stores; and finally a typescript file that contains the models of the objects needed in the code. All in all, this amounts to one Angular component and one Angular service.

In essence, an Angular component is a grouping of html, css, and typescript together to handle some kind of web functionality. A service, on the other hand, can hold some kind of broad functionality that could be utilized by different components. This library is made up essentially of just one component, one service, and one typescript file for declarations of the models of data that will need to be used. In a way, this implementation is similar to a model-view-controller pattern, with the exception that the Angular component is handling both the view and the controller.

The reason why this implementation was chosen was because once the coding is complete, the way an Angular component can be used is by simply placing into the HTML markup a template similar to any html element. Once the store locator's library module has been imported, the Angular component can be called simply by placing the following line in the HTML:

```
<lib-StoreLocatorLibrary [stores]="stores" [apiKey]="apiKey"  
></lib-StoreLocatorLibrary>
```

The stores variable represents an array of stores, and the apiKey variable is just the user's google maps API key that they received from Google. So it is a very neat solution, because to literally add the store locator to their HTML, it is just a single line of code. The developer passes in their data, and their api key, and then the library takes care of the rest.

Going further into the implementation of the key features is an examination of the components typescript file. The store-locator-library.component.ts file declares the

StoreLocatorLibraryComponent, which is an Angular Component. This component has many things that it handles. It takes the stores and api key as inputs. It has references to the `google.maps.Map` and an array of `google.maps.Marker`. Outside of the Google Maps variables, it handles current search type, current location latitude and longitude, total pages for pagination, the current page, the results for page, the search radius, etc.

The main responsibilities of the component file are the map manipulation and handling of user input. First and foremost, it handles loading the Google Maps API. It does this by overloading the `ngOnChanges()` lifecycle hook. This is a clever hook to use, because `ngOnChanges()` fires once the input parameters are set. This way, the code that is run to load the google maps script is only run once we have stores and the api key set as our inputs. None of the Google Maps API features work without first asynchronously loading its script, so the component waits for this to finish and then initializes the google map.

The next big set of responsibilities for the component are handling user input. The `store-locator-library.component.html` file holds the HTML template that is loaded by the Angular Component. The typescript file has variables and functions that handle the user input, getting the user's current location, and the search click event. Once the user clicks search, the component asks the service to handle the search and outputs the paginated results.

The service that comes with the library handles all of the specific. This is the `store-locator.library.service.ts`. The service holds the array of all of the stores. When the component loads, it sets the stores in the service. The service handles all of the searches. When a search is made, it handles figuring out whether stores are within the search radius from the search location.

Discussion

There were parts of the project that went very well, and there were also certain struggles towards the end with the actual packaging of the code into the reusable library. The parts of the project that went well were early on in the process. This included the research and design portion. Early on, when it wasn't yet decided how exactly development would be done, it was actually fairly easy to come up with the relevant design and frameworks to use. The decision to use Angular made the vision of creating a reusable component fairly straightforward. This is due to the very fact that Angular is a component based framework. So it became a natural choice.

The early development process had some problems with getting the Google Maps Api to work with Angular. This was somewhat paradoxical because Google manages Angular, so it seemed like it would be easy to get them to work together. It was actually somewhat of a headache to get the two integrated together in a way that was satisfactory. There were issues with getting certain libraries loaded correctly. Loading the Google Maps API script needed to be done differently than in the examples that were in the Google documentation since the Angular component lifecycle is a bit different than what was presented in the documentation.

It was in the late development process that the biggest problems arose. The issue arose while creating a working prototype from the library that was deployed to NPM. The whole process of translating the component from the dev environment into a library had many issues along the way. This was due to certain modules used during the development process not being able to be packaged in a library. Substitutions had to be made to use different modules that could be packaged. The main issue that arose was related to different dependency versions. Each Node.JS project has a package.json file that has all of the dependencies to other packages that are installed. So in this project, the big dependencies were for Bootstrap and the Google Maps Types. Those packages had been installed during development, and needed to be included as

dependencies in the library. In addition to these dependencies, the general dependencies for Angular modules such as the CommonModule and FormsModule needed to be included.

The initial development of the application was done using Visual Studio 2019, and the Angular application was bootstrapped using the Angular project template for a ASP.Net Web Application. This created a project that had an Angular version of 8. Later on, the library was created using the Angular Command Line Interface, which had an installed version of 10. Because of this version mismatch, an error occurred where the Angular dependency injection was throwing an error. In typical software development fashion, this error did not include information about the version type mismatch. The error itself was a NullInjector error, which really isn't helpful since the main issue wasn't a missing provider, which is what that error typically refers to, but instead just a version mismatch error. It took quite a lot of research and googling to figure out exactly how to fix this error. It took recreating a new development environment from the command line interface, and essentially starting from scratch and rebuilding everything with the same Angular version. The result of this was a working package, thankfully.

In closing, the process of developing an idea, designing the layout, writing the code to handle the main features, and finally deploying the application was a very rewarding experience. It demonstrated that not everything goes according to plan. There were many unforeseen complications that slowed down the development process, all the way from the beginning to the very end. But in working on these complications, it was very rewarding for the student developer to find the solution and create a working product.

References

Wandosell, G., Parra-Merono, M. C., & Banos, R. (n.d.). Online Store Locator: An Essential Resource for Retailers in the 21st Century.

Appendix A

Usability Testing Feedback

Jordan (Software Engineer/Architect):

Using the app:

Jordan: "Initial thoughts are that it looks nice."

Jordan: "Is this plug and play? Am I a store owner that can just add this to my website?"

Student: Yes, it is going to be a NPM package that developers can install and use the Angular component.

Jordan: "Nice. What parts of the page are the component and what parts of the page would be the website? Are there any search results yet? Are there any pins on the screen? What do the pins look like? Can I click on them? Is there mobile responsiveness?"

"Looks good after the search results. I can see the list result for mobile responsiveness."

"Top bar can be more mobile friendly."

"At a certain minimum width, can stack the map and the list results so the map doesn't get too small."

"What is happening when we move to the next page? Are the new pins the ones in the list display?"

Student: Yes.

"Good. Cool."

"On a mobile device, if you can click on a phone number to make it call the store."

"Directions looks good. Very cool."

Viewing the code:

Jordan: "So my understanding is you are passing an array of objects to the component?"

Student: Yes.

Jordan: "Recommend that you have documentation. What does your store object look like when using this? What if I really like this component but I want to customize things like button color or color scheme? Are there css classes I can set or modify? Can I pass styles into the component?"

Student: There are two ways. Either we make it open source and you can modify it yourself, or we can implement themes and have them pick the theme they want to use.

Jordan: "So when you update the reverseGeocodedAddress object the page automatically updates?"

Student: Yes.

Jordan: "Everything that is in the interpolation, these are all variables that come from the component?"

Student: Yes.

Suggestion: "When loading, if the map already exists in the ngOnChanges method, delete it so it can re-create."

Suggestion: "Consider moving reusable code into utils."

Saeed (Software Engineer):

Saeed: "How do you pass the data to it? How do you pass the json?"

Student: The data is not necessarily in json format. It can be from any data source. It is up to the developer to figure out how they will provide the data. The end result is it is in an array of the store object type.

Saeed: "So it just needs to be an array?"

Student: Yes.

Saeed: "Are you going to provide any way of creating the array? What if they don't have their store data in your format? Are you going to provide a way to create the data? How is this demo getting its data?"

Student: It is using a json file. It doesn't necessarily need to be from json though. In my case I used the Angular Http Client to load the data from a local json file. But they could load it from a server from a database if they were storing the data that way.

Saeed: "My suggestion would be to provide a way to generate a json file in your store format so that they can make sure it all works together. Some sort of console application would be a good addition. Something where you can type in store information, such as address, name, phone number, and then add it to a list, and then generate a json file as output. This way if they don't have their store information in a database they could generate it this way, and you make sure it fits your format."

Student: I think this is a good suggestion. Maybe outside the scope of my class, but definitely something that could be added on later. Do you have any comments about the app itself?

Saeed: "The map looks good. You need to finish the other search options. I think the map works good. It looks good over all. I can see all the results. Directions works fine. I think it all works fine."

Jane (Graphic Designer)

Jane: "I like how it looks. The fonts look good, the sizing looks good. The map looks good, and the positioning of the elements all look fine to me. It is clear how it functions. The dropdowns need to be finished. You need to get that working. I think the pagination section looks good. Is this the only color scheme that they can have?"

Student: Right now, yes.

Jane: "Okay. I think you should think about additional features. Like they should be able to customize the colors of the site. Some companies might want different colors based upon what their brand is. Do these windows only show this information? Can they add more information?"

Student: Right now that is all it supports.

Jane: "Okay. Look into ways of changing what information is displayed. For example, some people might want to know what kind of things are sold at a certain store. Or maybe the store owners want to display extra information like a sale."